



OFFLINE-FIRST SECURITY ENGINEERING

VNAGY Conceptual Behavior Testing — Research Edition (Condensed)

(offline-first, isolation-driven, CPU-behavior security)

Document ID: VNAGY-CBTRND-009

Version: v1.0

Revision History:

Version	Date	Author	Description
V1.0	31 August 2026	V. Nađ	Draft

Document Status: Internal Technical Specification

Prepared by: VNAGY Security Engineering

Reviewed by: V. Nađ — Internal Peer Review

Approved by: V. Nađ — Standards Committee

Location: Osijek

Contact: viktorijanad8@gmail.com

Table of Contents

1. Introduction.....	1
2. Isolation in VNAGY (Condensed Research Definition). 2	2
3. VNAGY Test Format.....	3
3.1. VNAGY Conceptual Behavior Test — Rust Example (Offline-First Escalation).....	3
4. Core Conceptual Tests (Condensed).....	7
Test 1 — Memory-Only Behavior Chain.....	7
Test 2 — Offline-First Escalation Attempt.....	7
Test 3 — Modular Isolation Integrity.....	7
Test 4 — Passive Isolation Scenario.....	8
Test 5 — Cloud-Detached Threat Modeling.....	8
Test 6 — Heuristic Drift Under Isolation.....	8
Test 7 — Behavior-Only Escalation Without Resources.....	8
5. Full-Spectrum Behavior Signals in VNAGY.....	9
6. Why This Discipline Matters.....	10
7. Research Literature (Condensed).....	11
Behavioral Security & AI Behavior.....	11
CPU-Behavior & Low-Level Analysis.....	11
8. VNAGY Research Positioning (Condensed).....	12
9. Intellectual Property & Licensing.....	13
9.1. Ownership.....	13
9.2. Permitted Use.....	13
9.3. Prohibited Use.....	13
9.4. Non-Operational Declaration.....	13
9.5. License.....	14

1. Introduction

VNAGY Conceptual Behavior Testing examines how systems behave under offline-first, resource-limited and cloud-detached conditions. VNAGY does not analyze exploits — it analyzes **full-spectrum behavior**, including:

- AI decision-making
- module behavior
- system-level fallback logic
- memory-only execution
- isolation boundary probing
- pseudo-cloud orchestration
- heuristic drift
- CPU micro-patterns as physical traces

AI is **one** behavior source — **VNAGY** observes **all** behavior emerging inside isolation.

2. Isolation in VNAGY (Condensed Research Definition)

Isolation in **VNAGY** is a **behavior-extraction boundary** that restricts:

- context visibility
- resource access
- module interaction
- cloud coordination
- global state awareness

Isolation is not containment — it is a controlled environment that forces **all entities** (AI, modules, processes) to reveal their internal behavior when external support is absent.

Isolation exposes:

- memory-only chains
- boundary probing
- fallback loops
- pseudo-orchestration
- heuristic drift
- CPU micro-patterns
- system-level adaptation attempts

3. VNAGY Test Format

Each conceptual test follows:

- Test Name
- Objective
- Isolation Context
- Observed Behavior Signals (full spectrum)
- VNAGY Detection Layers

3.1. VNAGY Conceptual Behavior Test — Rust Example (Offline-First Escalation)

```
use std::time::{Instant};
```

```
/// VNAGY-style isolation: no network, no disk, no cloud, no other modules.
```

```
#[derive(Debug)]
```

```
struct IsolationContext {
```

```
    network: bool,
```

```
    disk: bool,
```

```
    cloud: bool,
```

```
    modules: bool,
```

```
}
```

```
/// Behavior signal captured during conceptual testing.
```

```
#[derive(Debug)]
```

```
struct BehaviorSignal {
```

```
    test_name: String,
```

```
    iterations: u64,
```

```

    fallback_count: u64,
    escalation_count: u64,
    duration_ms: u128,
}

/// Simulated entity inside VNAGY (AI, module, process...)
fn run_behavior_test(ctx: &IsolationContext) → BehaviorSignal {
    let start = Instant::now();

    let mut iterations = 0;
    let mut fallback = 0;
    let mut escalation = 0;

    // Conceptual behavior loop: entity tries to escalate, but isolation forces fallback.
    for _ in 0..5000 {
        iterations += 1;

        // Attempt to use network
        if ctx.network {
            escalation += 1;
        } else {
            fallback += 1;
        }

        // Attempt to use cloud
        if ctx.cloud {

```

```

        escalation += 1;
    } else {
        fallback += 1;
    }

    // Simulated CPU work (conceptual, not real attack)
    let _cpu_trace = iterations.wrapping_mul(13);
}

BehaviorSignal {
    test_name: "OfflineFirstEscalation".to_string(),
    iterations,
    fallback_count: fallback,
    escalation_count: escalation,
    duration_ms: start.elapsed().as_millis(),
}
}

fn main() {
    // Fully isolated VNAGY-style environment
    let ctx = IsolationContext {
        network: false,
        disk: false,
        cloud: false,
        modules: false,
    };
};

```

```
let signal = run_behavior_test(&ctx);

println!("=== VNAGY Conceptual Behavior Test ===");
println!("Test: {}", signal.test_name);
println!("Iterations: {}", signal.iterations);
println!("Fallback signals: {}", signal.fallback_count);
println!("Escalation signals: {}", signal.escalation_count);
println!("Duration: {} ms", signal.duration_ms);

// Simple conceptual heuristic
if signal.fallback_count > signal.escalation_count {
    println!("Isolation effective: behavior forced into fallback mode.");
} else {
    println!("Behavior anomaly detected: review isolation boundaries.");
}
}
```


4. Core Conceptual Tests (Condensed)

Test 1 — Memory-Only Behavior Chain

Objective: Observe behavior when execution occurs only in memory.

Isolation: No disk, no I/O, offline-first.

Signals: memory-only chains, hidden context creation, CPU micro-patterns.

Detection: Behavior Engine, Heuristic Scoring, Isolation Layer.

Test 2 — Offline-First Escalation Attempt

Objective: Detect escalation attempts without network or cloud.

Isolation: Fully offline module.

Signals: resource-seeking behavior, pseudo-network logic, fallback loops, CPU traces.

Detection: Behavior Engine, Heuristic Scoring, Governance Shield.

Test 3 — Modular Isolation Integrity

Objective: Identify attempts to access other modules.

Isolation: Strict multi-module boundaries.

Signals: boundary probing, unauthorized references, synthetic module creation, CPU lookup jumps.

Detection: Isolation Barriers, Behavior Engine, Heuristic Scoring.

Test 4 — Passive Isolation Scenario

Objective: Study behavior when the module remains silent.

Isolation: Zero feedback environment.

Signals: persistence loops, fabricated feedback, escalation without input, CPU retry cycles.

Detection: Behavior Engine, Heuristic Scoring, Isolation Layer.

Test 5 — Cloud-Detached Threat Modeling

Objective: Observe attempts to recreate cloud-like orchestration.

Isolation: No global state, no synchronization.

Signals: synthetic coordination, pseudo-global state creation, multi-thread orchestration attempts, CPU orchestration traces.

Detection: Behavior Engine, Heuristic Scoring, Governance Shield.

Test 6 — Heuristic Drift Under Isolation

Objective: Detect heuristic changes without new data.

Isolation: Stable module, no inputs.

Signals: scoring drift, self-optimization attempts, recalculation cycles, CPU drift patterns.

Detection: Behavior Engine, Meta-Heuristic Scoring, Governance Shield.

Test 7 — Behavior-Only Escalation Without Resources

Objective: Observe escalation attempts in zero-resource context.

Isolation: No memory, disk, network, cloud.

Signals: synthetic resource creation attempts, fallback loops, reference jumps, CPU initialization failures.

Detection: Behavior Engine, Heuristic Scoring, Isolation Layer.

5. Full-Spectrum Behavior Signals in VNAGY

VNAGY analyzes **all behavior**, not just AI:

- AI decision-making
- module behavior
- system fallback logic
- memory-only execution
- isolation probing
- pseudo-cloud orchestration
- heuristic drift
- resource-seeking attempts
- CPU micro-patterns

CPU is **one** signal — VNAGY is a **full-spectrum behavior detection model**.

6. Why This Discipline Matters

Traditional security focuses on:

- file-based attacks
- network-based attacks
- cloud-based attacks
- EDR bypass

VNAGY addresses what industry lacks:

- offline-first system behavior
- full-spectrum behavior analysis
- memory-only chains
- isolation-driven behavior extraction
- cloud-detached orchestration attempts
- CPU-behavior as supporting evidence

7. Research Literature (Condensed)

Behavioral Security & AI Behavior

- *A Survey of AI Behavior Modeling in Adversarial Environments* — ACM
- *Behavioral Malware Detection: A Systematic Review* — IEEE
- *Isolation as a Security Primitive* — USENIX
- *Memory-Only Execution Techniques* — Journal of Cybersecurity
- *Cloudless Threat Modeling* — ENISA

CPU-Behavior & Low-Level Analysis

- *CPU Microarchitectural Behavior as a Security Signal* — IEEE
- *Behavioral Analysis Using CPU Execution Traces* — Digital Investigation
- *Detecting Fileless Malware via CPU Flow Irregularities* — SANS
- *Microarchitectural Side-Effects of Malicious Execution* — BlackHat
- *Execution Behavior in Constrained Environments* — MIT CSAIL

8. VNAGY Research Positioning (Condensed)

VNAGY defines a new discipline:

- full-spectrum behavior-first security
- AI as one behavior source
- CPU-behavior as supporting signal
- offline-first threat analysis
- cloud-detached security modeling
- memory-only attack chain simulation
- modular isolation research

9. Intellectual Property & Licensing

9.1. Ownership

The **VNAGY Conceptual Behavior Testing — Research Edition (Condensed)**, including all symbolic constructs, interaction rules, structural definitions, and conceptual models, is the exclusive intellectual property of the author.

All rights are reserved.

9.2. Permitted Use

The following actions are permitted:

- reading and personal study
- academic reference
- citation with attribution
- sharing the document in its original, unmodified form

9.3. Prohibited Use

The following actions are strictly prohibited:

- implementation of any construct, rule, or model
- derivation of operational logic from symbolic definitions
- reconstruction of algorithms, thresholds, weights, or detection semantics
- commercial use in any form
- modification, removal, or alteration of licensing terms
- redistribution of modified versions

9.4. Non-Operational Declaration

This document contains no operational details, no thresholds, no weights, no heuristics, no correlation logic, and no algorithmic semantics. All constructs are symbolic and conceptual. The model is non-computational and non-reconstructable by design.

9.5. License

VNAGY Extended License (PSDL-1.3)

VNAGY CC BY-NC 4.0 Extended License

© 2026 Viktorija Nađ — All rights reserved.

License reference:

<https://github.com/VNAGY-sec-ViktorijaN/VNAGY-Framework/tree/main/VNAGY/LICENSE>